

Pageflow UI

Every export, hook, component, option & method — with signatures and examples.

The exhaustive client-side API reference for `@pageflow/core` and `@pageflow/react`. For task-oriented recipes see [PAGEFLOW_USAGE.pdf](#); for architecture & the wire protocol see [PAGEFLOW_GUIDE.pdf](#). Everything below is imported from `@pageflow/react` unless noted.

Import map: most apps import from `@pageflow/react`, which re-exports the core utilities. Only Vue/Svelte adapters or Node SSR use `@pageflow/core` directly.

1. Bootstrap

`createPageflowApp(options)`

Boots the app. Call once in your entry file. Handles CSR and SSR.

```
createPageflowApp({ resolve, setup, title?, progress?, id?, page?, render? }): Promise<...>
```

Option	Type	Meaning
<code>resolve</code>	<code>(name) => Component Promise</code>	Map a component name to its module. Usually via <code>resolvePageComponent</code> .
<code>setup</code>	<code>(({el, App, props}) => ...)</code>	Mount the app (e.g. <code>createRoot(el).render(<App {...props}/>)</code>).
<code>title</code>	<code>(title) => string</code>	Transform the document title (e.g. append " · MyApp").
<code>progress</code>	<code>false {delay,color,includeCSS,showSpinner}</code>	Top progress bar config, or <code>false</code> to disable.
<code>id</code>	<code>string</code>	Root element id (default <code>'app'</code>). Must match <code>PAGEFLOW_APP_ID</code> .
<code>page</code> , <code>render</code>	—	SSR only: initial page + <code>renderToString</code> .

`resolvePageComponent(path, pages)`

`@pageflow/core` Resolve a component name (or array of candidates) against a glob map; awaits lazy loaders.

```
resolvePageComponent<T>(path: string | string[], pages: PagesMap<T>): Promise<T>
```

`installCsrfAutoRefresh(options?) · registerPageflowSW(options?)`

Call once at bootstrap. See §7 (CSRF) and §8 (offline).

2. router

The single navigation engine (same instance across the app). All methods return `void` and fire the lifecycle callbacks/events.

Method	Signature
<code>get/post/put/patch</code>	<code>(url, data = {}, options?: VisitOptions)</code>
<code>delete</code>	<code>(url, options?: VisitOptions)</code>
<code>visit</code>	<code>(url, options?: VisitOptions)</code> — the general form
<code>reload</code>	<code>(options?: ReloadOptions)</code> — re-fetch current URL (async, no scroll change)
<code>replace / push</code>	<code>(ClientSideVisitOptions)</code> — client-only page mutation (no server hit)
<code>poll</code>	<code>(intervalMs, reloadOptions?, {autoStart,keepAlive}) → {start,stop}</code>
<code>prefetch</code>	<code>(url, options?, {cacheFor})</code>
<code>cancel / cancelAll</code>	abort in-flight visit(s)
<code>flush / flushAll</code>	drop prefetch cache entry / all
<code>remember / restore</code>	<code>(data, key?) / (key?)</code> — history-backed local state
<code>on</code>	<code>(event, cb) => () => void</code> — subscribe to a global event (§9)

Note: `post/put/patch/delete` default `preserveState: true` so form state & validation errors survive the round-trip.

VisitOptions — the full option set

Option	Type / default	Meaning
<code>method</code>	<code>'get' 'post' 'put' 'patch' 'delete'</code>	HTTP verb.
<code>data</code>	<code>object FormData</code>	Body (or query for GET). Files auto-convert to FormData.
<code>only / except</code>	<code>string[] = []</code>	Partial reload: include / exclude prop keys.
<code>reset</code>	<code>string[] = []</code>	Reset these merged/paginated props server-side.
<code>headers</code>	<code>Record<string,string></code>	Extra request headers.
<code>preserveScroll</code>	<code>bool 'errors' (page)=>bool = false</code>	Keep scroll position.
<code>preserveState</code>	<code>bool 'errors' (page)=>bool</code>	Keep component state (no remount).

replace	bool = false	Replace history entry instead of pushing.
errorBag	string null	Namespace for validation errors (multiple forms).
forceFormData	bool = false	Always send multipart, even without files.
queryStringArrayFormat	'brackets' 'indices'	Array encoding in query strings.
async	bool = false	Non-blocking visit (doesn't cancel other async visits).
showProgress	bool	Show the top progress bar (default: not for async).
fresh / preserveUrl / loadPage	bool	Advanced: skip cache / keep URL / suppress component swap.

Lifecycle callbacks (on any visit)

```
onBefore(visit) · onStart(visit) · onProgress(event) · onSuccess(page) · onError(errors) · onCancel() · onFinish(visit) · onCancelToken({cancel})
```

onBefore returning false cancels the visit. Fired in order; onFinish always runs.

3. Components

<Link>

Smart `<a>` (or any element via `as`) that navigates the Pageflow way.

Prop	Type	Meaning
<code>href</code>	<code>string {url, method}</code>	Target (required).
<code>method</code>	Method	Verb (default get).
<code>data</code>	object	Payload / query.
<code>as</code>	<code>ElementType</code>	Render as (e.g. <code>'button'</code>).
<code>only / except</code>	<code>string[]</code>	Partial reload.
<code>preserveScroll / preserveState / replace / async</code>	—	Same as VisitOptions.
<code>prefetch</code>	<code>bool 'hover' 'mount' 'click' []</code>	Pre-fetch strategy. <code>true</code> = hover.
<code>cacheFor</code>	<code>number string</code>	Prefetch TTL (default 30000ms).
<code>headers</code> , <code>queryStringArrayFormat</code> , <code>all on*</code>	—	Passed through to the visit.

```
<Link href="/users" prefetch="hover">Users</Link>
<Link href="/logout" method="post" as="button">Log out</Link>
<Link href="/reports" only={['summary']} preserveScroll>Refresh</Link>
```

<Form>

Declarative form — serializes native fields (nested `user[name]` , arrays, files) and submits.

Prop	Type	Meaning
<code>action</code>	string	Endpoint URL.
<code>method</code>	Method	Verb (default get).
<code>resetOnSuccess / resetOnError</code>	<code>bool string[]</code>	Reset all/named fields after.
<code>validateOn</code>	<code>'blur' 'change'</code>	Live precognition validation.
<code>validationDebounce</code>	number = 300	Debounce for <code>validateOn="change"</code> .
<code>transform</code>	<code>(data)=>data</code>	Mutate before send.
<code>only / except / headers / preserveScroll / errorBag / forceFormData</code>	—	Visit options.
<code>on*</code>	—	onBefore/onStart/onProgress/onSuccess/onError/onFinish/onCancel.
<code>children</code>	<code>ReactNode (props)=>ReactNode</code>	Render-prop gets <code>{errors, processing, wasSuccessful, reset, submit, setError, clearErrors}</code> .

<Head>

Manage `<title>` and meta tags per page. Props: `title?: string` , `children` . Use `head-key` on a child to dedupe/replace a specific tag.

```
<Head title="Dashboard">
  <meta name="description" content="..." head-key="desc" />
</Head>
```

<Can>

Prop	Type	Meaning
<code>permission</code>	string	Require this permission.
<code>anyOf / allOf</code>	<code>string[]</code>	Require any / all.
<code>role</code>	string	Require a role.
<code>fallback</code>	ReactNode	Rendered when denied (default nothing).

UX gating only — the server Service layer is the real authorization boundary.

<Deferred> · <WhenVisible>

```
<Deferred data="report" fallback=<Spinner />>{/* renders once loaded */}</Deferred>
<WhenVisible data="comments" buffer={200} fallback=<Spinner /> always={false}>{...}</WhenVisible>
```

Deferred : `data` (string|string[]), `fallback` . WhenVisible : `data` or `params` , `buffer` (px), `as` , `always` , `fallback` .

4. Data & form hooks

usePage<T>()

```
usePage<T> = PageProps>(): Page<T> // .props, .component, .url, .version
```

```
const { users, auth } = usePage<{ users: User[]; auth: Auth }>().props
```

useForm(initial) / useForm(rememberKey, initial)

Returns a form object. With a `rememberKey`, data+errors persist across navigation.

Property	Type
<code>data</code>	the form values
<code>errors</code> / <code>hasErrors</code>	field → message / bool
<code>processing</code>	bool (in-flight)
<code>progress</code>	upload progress or null
<code>isDirty</code>	changed vs defaults
<code>wasSuccessful</code> / <code>recentlySuccessful</code>	bool (recently = 2s window)

Method	Signature
<code>setData</code>	<code>(key, value) · (obj) · (prev => obj)</code>
<code>get/post/put/patch</code>	<code>(url, options?)</code>
<code>delete</code>	<code>(url, options?)</code>
<code>submit</code>	<code>(method, url, options?)</code> or <code>({url,method}, options?)</code>
<code>reset</code>	<code>(...fields)</code> — to defaults (all if none named)
<code>setError</code> / <code>clearErrors</code>	<code>(field,msg) / (...fields)</code>
<code>setDefaults</code>	<code>() (field,val) (obj)</code>
<code>transform</code>	<code>(cb: (data)=>object)</code>
<code>replaceData</code> / <code>resetAndClearErrors</code> / <code>cancel</code>	—

Submit options (on top of VisitOptions): `resetOnSuccess`, `resetOnError` (`bool | string[]`).

usePrecognition(method, url, getData, options?)

Server-truth live validation without side effects. Debounced (default 300ms).

```
usePrecognition(method, url, () => data, { debounce? }) → { errors, hasErrors, validating, validate, validateField, clearErrors, setErrors }
```

```
const pre = usePrecognition('post', '/register', () => data)
<input onBlur={() => pre.validateField('email')} />
```

5. Realtime & UX hooks

useReactiveProps(keys, options?)

```
useReactiveProps(keys: string | string[], { channel?, endpoint?, params?, debounce?, reload?, onError? })
```

Subscribes to the SSE channel; on a stale signal for a watched key, runs an authorized `router.reload({only})`. Options: `channel` (topic), `endpoint` (default `/pageflow/stream`), `params` (scoping, e.g. a resource id), `debounce` (default 50), `reload` (extra reload options).

```
useReactiveProps(['orders'], { channel: 'dashboard' })
```

useAuth()

```
useAuth(): { userId, tenantId, roles, permissions, authenticated, can, canAny, canAll, hasRole }
```

useDirtyGuard(dirty, options?)

Warns before leaving a form with unsaved changes (browser unload + in-app nav). Options: `message`, `guardUnload`.

```
const form = useForm({...}); useDirtyGuard(form.isDirty)
```

usePoll(interval, reloadOptions?, options?)

Auto re-fetch; pauses in background tabs. Options `{autoStart, keepAlive}`. Returns `{start, stop}`.

usePrefetch(options?)

Reactive prefetch status for the current URL → `{ lastUpdatedAt, isPrefetching, isPrefetched, flush }`.

useRemember(initial, key?)

Like `useState` but persisted in history state (survives back/forward & navigation).

useFlushOnIdentityChange()

Place once near the app root — purges the prefetch cache AND service-worker cache whenever the identity/tenant changes (login/logout/switch). Security-relevant on shared devices.

6. CSRF utilities

@pageflow/core (re-exported from react).

Export	Signature	Use
installCsrfAutoRefresh	(({endpoint?})	axios interceptor: on a CSRF 403, refresh token & retry once (same-origin only).
configureCsrf	(({headerName?, metaName?, resolver?})	Override how the token is read (custom header/meta/resolver).
csrfToken	(): string null	Current token (from the meta tag by default).
setCsrfToken	(token)	Write the token (updates the meta tag).
csrfHeaderName	(): string	The header name (default <code>X-CSRF-Token</code>).
isSameOriginUrl	(url): boolean	Same-origin guard (fails closed). Used internally to never leak the token cross-origin.

```
// custom scheme (e.g. token in a shared prop)
configureCsrf({ headerName: 'X-XSRF-TOKEN', resolver: () => store.token })
```

7. Offline (service worker)

Export	Signature	Use
registerPageflowSW	(({url?, scope?})	Register <code>pageflow-sw.js</code> (default <code>/pageflow-sw.js</code> , scope <code>/</code>).
clearPageflowSWCache	(): Promise<void>	Wipe the SW page cache (call on logout).

Caching is opt-in: only pages the server marks (`X-Pageflow-Cache: 1` / `Cache-Control: public`) are cached. Static assets are cache-first automatically.

8. Low-level utilities

Export	Signature	Use
precognitiveValidate	(method, url, data, only?) → Promise<Errors>	One-shot validation (the primitive behind usePrecognition).
normalizeErrors	(body) → Errors	Coerce an error envelope to <code>{field: message}</code> .
openReactiveStream	(({endpoint, channel?, params?, onStale, onError?}) → dispose()	Low-level SSE subscription (behind useReactiveProps).
supportsReactive	(): boolean	EventSource availability.
router	—	The navigation engine (§2).

9. Global events

Subscribe with `router.on(event, cb) → unsubscribe()`. Cancelable events cancel when the callback returns `false`.

Event	Payload (<code>e.detail</code>)	Cancelable
before	<code>{ visit }</code>	yes
start	<code>{ visit }</code>	no
progress	<code>{ progress }</code>	no
success	<code>{ page }</code>	no
error	<code>{ errors }</code>	no
invalid	<code>{ response }</code> (non-Pageflow response)	yes
exception	<code>{ exception }</code>	yes
finish	<code>{ visit }</code>	no
navigate	<code>{ page }</code>	no
prefetching / prefetched	<code>{ visit } / { visit, response, fetchedAt }</code>	no

```
const off = router.on('error', (e) => toast(Object.values(e.detail.errors)[0]))
// later: off()
```

10. Layouts (persistent)

Give a page a persistent layout that doesn't remount on navigation:

```
function Users() { /* ... */ }
Users.layout = (page) => <AppShell>{page}</AppShell> // or an array to nest
export default Users
```

11. TypeScript

Run `hkm pageflow:types` to generate `pageflow.d.ts` (shared props + page registry). Then augment per page:

```
declare module '@pageflow/react' {  
  interface PageflowPages { 'Users/Index': { users: User[] } }  
}
```

Key types: `Page<T>`, `PageProps`, `Method`, `VisitOptions`, `ReloadOptions`, `Errors`, `Progress`, `PageflowLinkProps`, `PageflowFormProps`, `FormRenderProps`, `AuthHelpers`, `PageflowAuth`.

Pageflow UI Complete Reference · AlfacodeTeam PhpServicePlatform · companions: [PAGEFLOW_USAGE.pdf](#) (recipes), [PAGEFLOW_GUIDE.pdf](#) (architecture + security). Fork of Inertia.js (MIT).